Email2git: Matching Linux Code to its Mailing List discussion

alexcourouble.com/oss2017.html



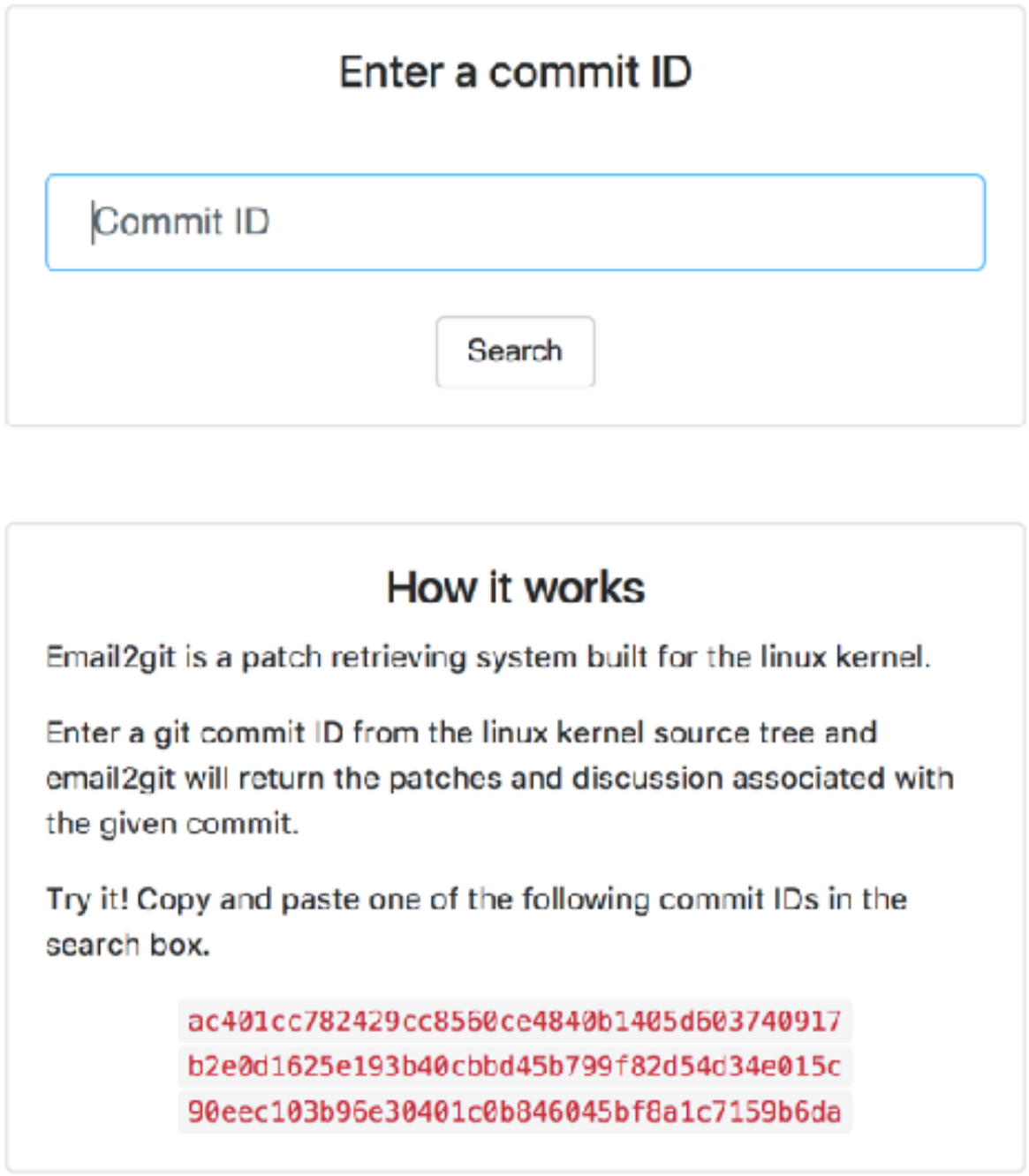
Alexandre Courouble, Bram Adams
Polytechnique Montreal

Kate Stewart
Linux Foundation

Daniel German
University of Victoria

Demo

What is Email2git?



Why?

- Linux **contribution process** doesn't provide traceability from **git** to **code review**
  - Mailing list
  - Patchwork

Use cases

- Allows **developers** to understand **design decisions**
  - Security
  - Bug fixing
  - New comers
  - <Your use case?>

Use cases

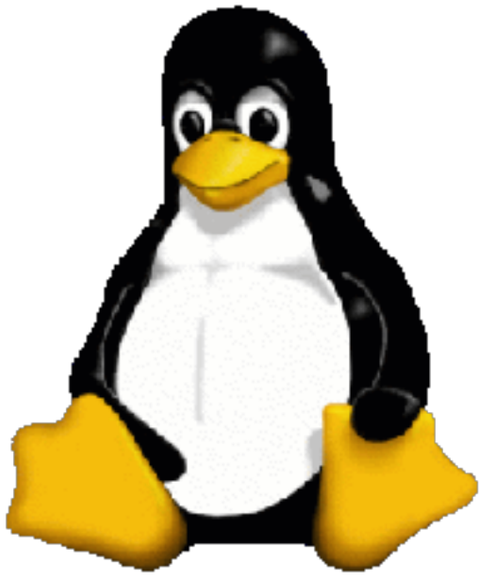


CHAOSS - Community Health Analytics OSS

Where does the data come from?

The Data

Two Sides



Linux git repository

git log -p



patchwork.kernel.org

MySQL queries

Patchwork

Patch tracking system



- **Extracts patches from Linux mailing lists**
- **Keeps track of code reviews**
- **User friendly interface**

cregit

```
int __ref create_proc_profile(void) /* false positive from hotcpu_notifier */
{
        struct proc_dir_entry *entry;
        int err = 0;

        if (!prof_on)
                return 0;

        cpu_notifier_register_begin();

        if (create_hash_tables()) {
                err = -ENOMEM;
                goto out;
        }

        entry = proc_create("profile", S_IWUSR | S_IRUGO,
                            NULL, &proc_profile_operations);
        if (!entry)
                goto out;
        proc_set_size(entry, (1 + prof_len) * sizeof(atomic_t));
        __hotcpu_notifier(profile_cpu_callback, 0);

out:
        cpu_notifier_register_done();
        return err;
}
```

Contributors

william lee irwin iii	william lee irwin iii	58	58.00%
srivatsa s. bhat	srivatsa s. bhat	26	26.00%
paolo ciarrocchi	paolo ciarrocchi	5	5.00%
david howells	david howells	4	4.00%
denis v. lunev	denis v. lunev	4	4.00%
al viro	al viro	2	2.00%
dave hansen	dave hansen	1	1.00%

- **Token-level git blame**
- **~95% accuracy**
- **git blame = ~80%**

How Does the Algorithm Work?

Will My Patch Make It? And How Fast?

Case Study on the Linux Kernel

Yujuan Jiang, Bram Adams
MCIS, Polytechnique Montréal, Canada
{yujuan.jiang,bram.adams}@polymtl.ca

Daniel M. German
University of Victoria, Canada
dmg@uvic.ca

***Abstract*—The Linux kernel follows an extremely distributed reviewing and integration process supported by 130 developer mailing lists and a hierarchy of dozens of Git repositories for version control. Since not every patch can make it and of those that do, some patches require a lot more reviewing and integration effort than others, developers, reviewers and integrators need support for estimating which patches are worthwhile to spend effort on and which ones do not stand a chance. This paper cross-links and analyzes eight years of patch reviews from the kernel mailing lists and committed patches from the Git repository to understand which patches are accepted and how long it takes those patches to get to the end user. We found that 33% of the patches makes it into a Linux release, and that most of them need 3 to 6 months for this. Furthermore, that patches developed by more experienced developers are more easily accepted and faster reviewed and integrated. Additionally, reviewing time is impacted by submission time, the number of affected subsystems by the patch and the number of requested reviewers.**

I. Introduction

Integration of code changes into a project's main repository is an open source developer's ultimate goal, since it marks the first step towards inclusion in an official product release. An open source project like the Linux kernel, for example, integrates between 8,000 and 12,000 patches in a new release, contributed by more than 1,000 developers [1]. These patches only represent the "lucky few". Studies on Apache and other open source systems have shown how only 40% of the patches considered for integration eventually succeed [2], [3], [4].

One of the major reasons for the relatively low success rate of integration is the complexity of this process. The patches first need to pass a gate-keeper who performs a review of the code [2], [5], [6], before the code is merged by an integrator (e.g., release engineer) into the corresponding branch of the open source project [7], [8], [9]. Code reviews fail when a patch does not implement a relevant, working feature or bug fix, or when the project's development guidelines are not followed [10]. The actual integration (merging) fails when the patch interacts incorrectly with other patches or the merging process creates too many merge conflicts [11]. In case of integration issues, the developer needs to go back to the drawing board and try to integrate the code again. In the worst case, a patch will be rejected time and time again until the developer eventually gives up.

As a result, the integration process looks like a black box to most developers, with unpredictable outcome. Everyone knows the stories of disgruntled developers, even experienced ones, whose changes did not make it after putting months of work into them (e.g., [12], [13]). Even major projects like the Google Android mobile platform have problems getting their Linux kernel modifications integrated into the official kernel version [11]. Yet, determining up front whether a patch will make it, and how long it will take, is a grey area. Research on code reviews has shown how small patches [6], [4] sent by experienced developers [2] are more likely to be accepted by the reviewers, but it is not clear if these characteristics play the same role during the actual integration of the patch with other patches. Similarly, the impact of these characteristics on the time it takes to get a patch into a release is unclear.

This paper studies the relation of patch characteristics with (1) the probability of acceptance into an official release and (2) the time between submitting a patch for review and acceptance. We also analyze if these relations change over time. Our empirical analysis is based on eight years of patch review data and version control data from the Linux kernel project, which is a 20 year-old, popular open source system containing more than 15 MLOC of source code. We address the following research questions:

RQ1) What percentage of submitted patches has been integrated successfully, and how much time did it take?

Around 33% of patches are accepted. Reviewing time has been dropping down to 1–3 months, while integration time steadily has been increasing towards 1–3 months, bringing the total time to 3–6 months.

RQ2) What kind of patch is accepted more likely?

Developer experience, patch maturity and prior subsystem churn play a major role in patch acceptance, while patch characteristics and submission time do not.

RQ3) What kind of patch is accepted faster?

Reviewing time is impacted by submission time, the number of affected subsystems, the number of suggested reviewers and developer experience, while integration time is impacted by the same attributes as patch acceptance.

First, we provide background about the Linux kernel integration process (Section II), followed by an explanation of our case study methodology (Section III). Section IV presents the results of our case study, followed by a discussion of threats to validity (Section V). We finish the paper with related work (Section VI) and the conclusion (Section VII).

Tracing Back the History of Commits in Low-tech Reviewing Environments

A case study of the Linux kernel

Yujuan Jiang,
Bram Adams
MCIS, Polytechnique
Montréal, Canada
{yujuan.jiang,
bram.adams}@polymtl.ca

Foutse Khomh
SWAT, Polytechnique
Montréal, Canada
foutse.khomh@polymtl.ca

Daniel M. German
University of Victoria, Canada
dmg@uvic.ca

ABSTRACT

Context: During software maintenance, people typically go back to the original reviews of a patch to understand the actual design rationale and potential risks of the code. Whereas modern web-based reviewing environments like gerrit make this process relatively easy, the low-tech, mailing list based reviewing environments of many open source systems make linking a commit back to its reviews and earlier versions far from trivial, since (1) a commit has no physical link with any reviewing email, (2) the discussed patches are not always fully identical to the accepted commits and (3) some discussions last across multiple email threads, each of which containing potentially multiple versions of the same patch.
Goal: To support maintainers in reconstructing the reviewing history of kernel patches, and studying (for the first time) the characteristics of the recovered reviewing histories.
Method: This paper performs a comparative empirical study on the Linux kernel mailing lists of 3 email-to-email and email-to-commit linking techniques based on checksums, common patch lines and clone detection.
Results: Around 25% of the patches had an (until now) hidden reviewing history of more than four weeks, and patches with multiple versions typically are larger and have a higher acceptance rate than patches with just one version.
Conclusion: The plus-minus-line-based technique is the best approach for linking patch emails to commits, while it needs to be combined with the checksum-based technique for linking different patch versions.

Categories and Subject Descriptors

D.2.7 [**Software Engineering**]: Distribution, Maintenance, and Enhancement—*Restructuring, reverse engineering, and reengineering*; D.2.9 [**Software Engineering**]: Management—*Software configuration management*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
ESEM'14 September 18-19, 2014, Torino, Italy.
Copyright 2014 ACM 978-1-4503-2774-9/14/09... ...$15.00.

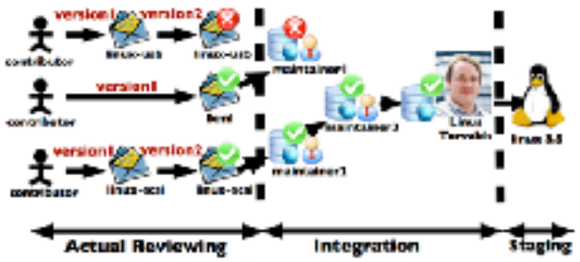


Figure 1: The reviewing and integration process of a patch in the Linux kernel project (adapted from [19]).

General Terms

Software Engineering

Keywords

review, traceability, clone detection, low-tech reviewing environment, mailing list, open source, Linux kernel

1. INTRODUCTION

In November 2003, there was a suspected backdoor attempt to taint the source code of the Linux kernel project [23]. The Linux team noticed that a patch popped up in the CVS copy of the version control system that never appeared in the BitKeeper master copy. This patch pretended to just check for errors, but actually contained a back door that could render a system vulnerable. Up until today, it is still not clear whether this code was inserted by a malicious hacker, since there was (and still is) no explicit link from a commit back to all emails reviewing that patch or earlier versions of it. The recent "heartbleed" vulnerability [16] also highlights the need to have traceability from commits in the version control system to patches in the mailing list. Full traceability would have made a quicker and more accurate audit of the code possible. Apart from security audits, traceability from submitted patches to accepted commits would also benefit regular maintenance, as it will make it easy to review the email messages around that patch which comprise the reviewing and design discussions of the patches [2][5][25].

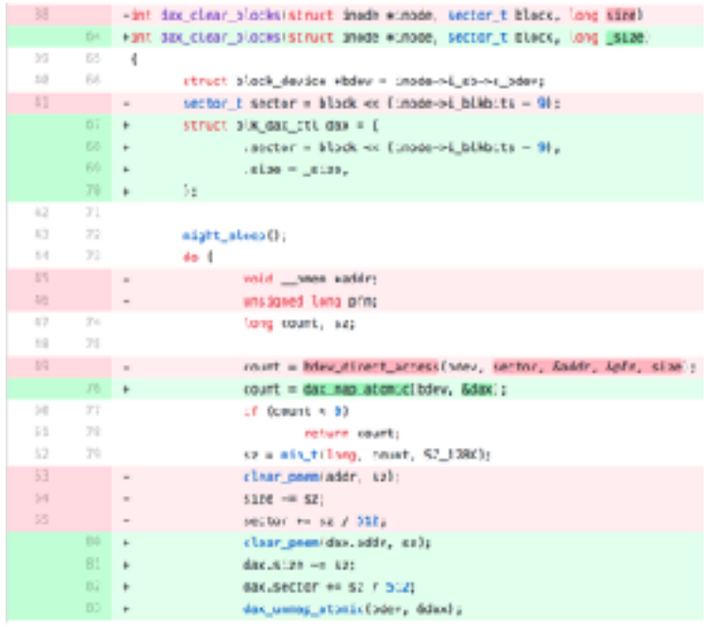
Although more and more open and closed source systems are migrating towards modern, online reviewing environments like Gerrit, which consolidate all patch versions, dis-

Original Algorithm

Git Log Output

Commit IDs

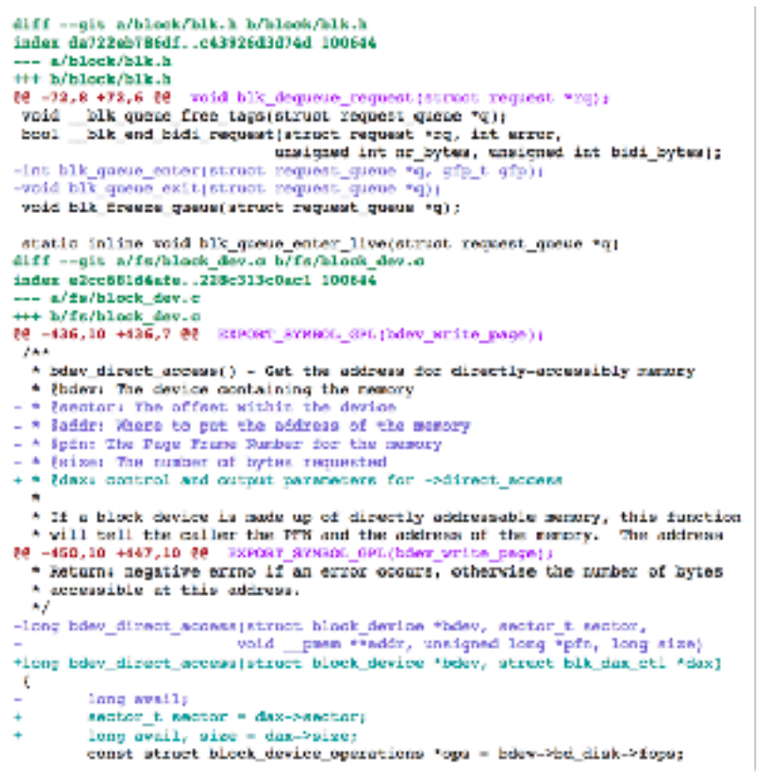
…
b3242dba9ff285962fe84d1135cafe9383d721f0
67a3b5cb33633f39db8809ae56c8c1752b541daa
17e34c4fd0be14e989b08734b302cd357126fe2d
42e6d5e5ee05a0a4fa6f71ffeeafc367a1483740
105065c3f7164d756ee5495b8670e57f2bcf65c3
8c7932a32e1ebf6e845bd32a5399cce442d2e745
d580e80c7f2ffb8f86e2417495692ce8bcc91dad
0b31c3ec1b0eac7ca71a7aa4a4a93d02fcade33d
1f5de42da44adbd4248f7ed6d1650eb1c53f87a0



Mailing Lists Patches

Patchwork Patch IDs

…
4693
2202
6649
2231
2203
2204
…



Comparing diff's +/- lines

@alexcourouble

Original Algorithm

Scaling problem

500k +
commits since
2009

1.4 M + patches
sent since 2009

Scaling the Algorithm

Narrowing down the search space

Heuristics

Email Subject — Commit Summary
Patch Author
Files Affected by Patch

C : Commit
P : Patch

C,C,C,C,C,C,C,C,C
P,P,P,P,P,P,P,P,P,P,P,P,P,P

Subject matching

Remaining commits/patches

Matches

C : P,P,P,P, …
C : P,P,P,P, …
C : P,P,P,P, …

Matched *commits/patches*

C,C,C,C,C
P,P,P,P,P,P,P,P,P,P

Author-map matching

Matches

Remaining commits/patches

Matches

Unmatched *commits/patches*

C,C,C
P,P,P,P,P,P

File name map matching

C,C
P,P,P,P

Step 1 : Email subject

Git Log

```
commit b2e0d1625e193b40cbbd45b799f82d54d34e015c
Author: Dan Williams <dan.j.williams@intel.com>
Date:   Fri Jan 15 16:55:59 2016 -0800

    dax: fix lifetime of in-kernel dax mappings with dax_map_atomic()

    The DAX implementation needs to protect new calls to ->direct_access()
    and usage of its return value against the driver for the underlying
    block device being disabled.  Use blk_queue_enter()/blk_queue_exit() to
    hold off blk_cleanup_queue() from proceeding, or otherwise fail new
    mapping requests if the request_queue is being torn down.

    This also introduces blk_dax_ctl to simplify the interface from fs/dax.c
    through dax_map_atomic() to bdev_direct_access().

    [willy@linux.intel.com: fix read() of a hole]
    Signed-off-by: Dan Williams <dan.j.williams@intel.com>
    Reviewed-by: Jeff Moyer <jmoyer@redhat.com>
    Cc: Jan Kara <jack@suse.com>
    Cc: Jens Axboe <axboe@fb.com>
    Cc: Dave Chinner <david@fromorbit.com>
    Cc: Ross Zwisler <ross.zwisler@linux.intel.com>
    Cc: Matthew Wilcox <willy@linux.intel.com>
    Signed-off-by: Andrew Morton <akpm@linux-foundation.org>
    Signed-off-by: Linus Torvalds <torvalds@linux-foundation.org>
```

Linux mailing lists

```
Subject: [PATCH 8/8] dax: fix lifetime of in-kernel dax mappings with
        dax_map_atomic()
From: Dan Williams <dan.j.williams@intel.com>
To: linux-nvdimm@lists.01.org
Date: Tue, 17 Nov 2015 12:16:35 -0800
```

This step finds patches for **~55%** of **commits**

Step 2 : Author name matching

Git Log

```
Author: Dan Williams <dan.j.williams@intel.com>

    dax: fix lifetime of in-kernel dax mappings with dax_map_atomic()

    The DAX implementation needs to protect new calls to ->direct_access()
    and usage of its return value against the driver for the underlying
    block device being disabled.  Use blk_queue_enter()/blk_queue_exit() to
    hold off blk_cleanup_queue() from proceeding, or otherwise fail new
    mapping requests if the request_queue is being torn down.

    This also introduces blk_dax_ctl to simplify the interface from fs/dax.c
    through dax_map_atomic() to bdev_direct_access().

    [willy@linux.intel.com: fix read() of a hole]
    Signed-off-by: Dan Williams <dan.j.williams@intel.com>
    Reviewed-by: Jeff Moyer <jmoyer@redhat.com>
    Cc: Jan Kara <jack@suse.com>
    Cc: Jens Axboe <axboe@fb.com>
    Cc: Dave Chinner <david@fromorbit.com>
    Cc: Ross Zwisler <ross.zwisler@linux.intel.com>
    Cc: Matthew Wilcox <willy@linux.intel.com>
    Signed-off-by: Andrew Morton <akpm@linux-foundation.org>
    Signed-off-by: Linus Torvalds <torvalds@linux-foundation.org>
```

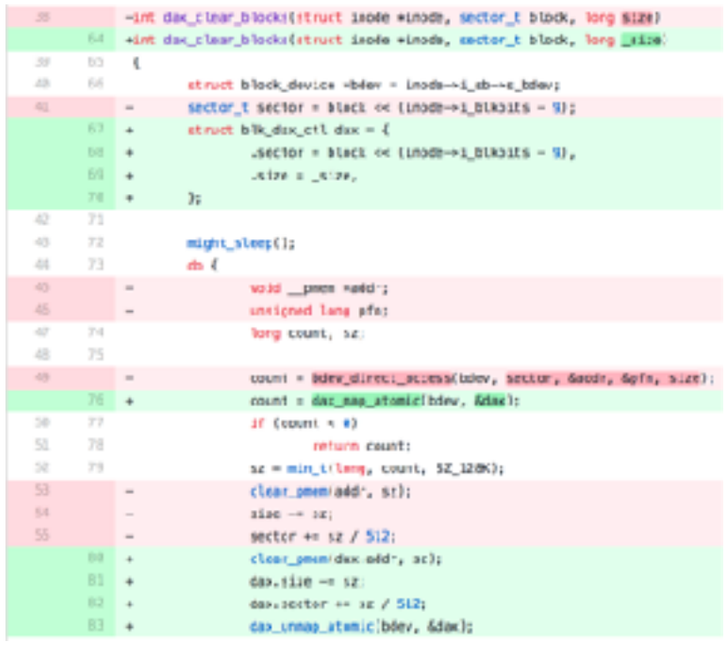
Linux mailing lists

```
Subject: [PATCH 8/8] dax: fix lifetime of in-kernel dax mappings with
        dax_map_atomic()
From: Dan Williams <dan.j.williams@intel.com>
To: linux-nvdimm@lists.01.org
Date: Tue, 17 Nov 2015 12:16:35 -0800
```

Step 2 : Author name matching

Commit ID : Author Name ⟶ **Author Name : Patches sent**

“b2e0d16…” ⟶ *“Dan Williams”* *“Dan Williams”* ⟶ *[Patch 1, Patch 2, Patch 3, …]*



+/- line comparison

Commit - patch matches

Step 3 and 4: File name / file path matching

git log --name-only --pretty=oneline

```
00baac21e4585f4258f919502c6e23f1e5edc98c mm: fix MADV_[FREE|DONTNEED] TLB flush miss problem
arch/arm/include/asm/tlb.h
arch/ia64/include/asm/tlb.h
arch/s390/include/asm/tlb.h
arch/sh/include/asm/tlb.h
arch/um/include/asm/tlb.h
include/asm-generic/tlb.h
include/linux/mm_types.h
mm/memory.c
0a2da20ce64e7f1d8b5bb30ab4ae3e0e3d3a6fcd mm: make tlb_flush_pending global
include/linux/mm_types.h
mm/debug.c
56236a59556cfc3bac7bffb7e5f433b5cf0af880 mm: refactor TLB gathering API
arch/arm/include/asm/tlb.h
arch/ia64/include/asm/tlb.h
arch/s390/include/asm/tlb.h
arch/sh/include/asm/tlb.h
arch/um/include/asm/tlb.h
include/asm-generic/tlb.h
include/linux/mm_types.h
mm/memory.c
```

Patch content

```
diff --git a/include/linux/blkdev.h b/include/linux/blkdev.h
index 3fe27f8d91f0..8aa53454ce27 100644
--- a/include/linux/blkdev.h
+++ b/include/linux/blkdev.h
```

Step 3 and 4: File name / file path matching

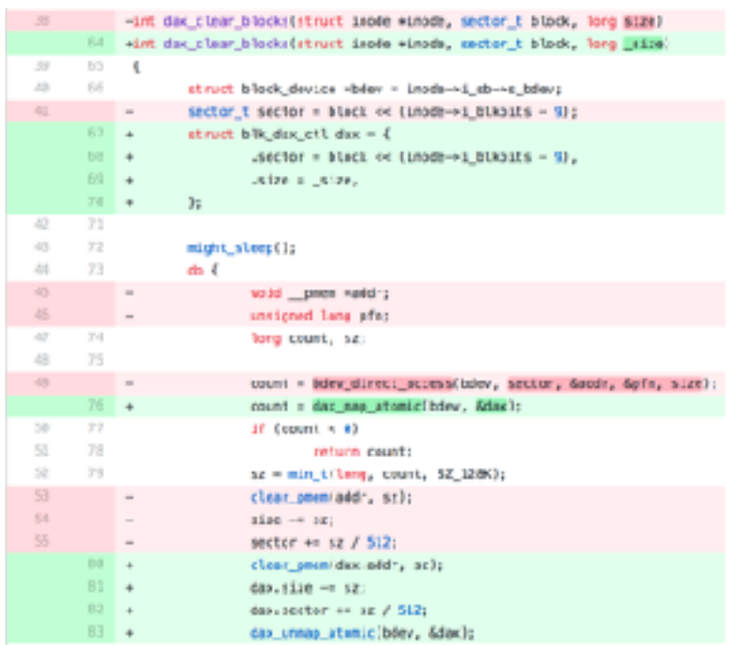
Commit ID : Files touched → **File name/path : Patches**

fs/dax.c
include/linux/blkdev.h
fs/block_dev.c

“b2e0d16…” → *“fs/dax.c”* → *[Patch 1, Patch 2, Patch 3, …]*

“include/linux/blkdev.h” → *[Patch 1, Patch 2, Patch 3, …]*

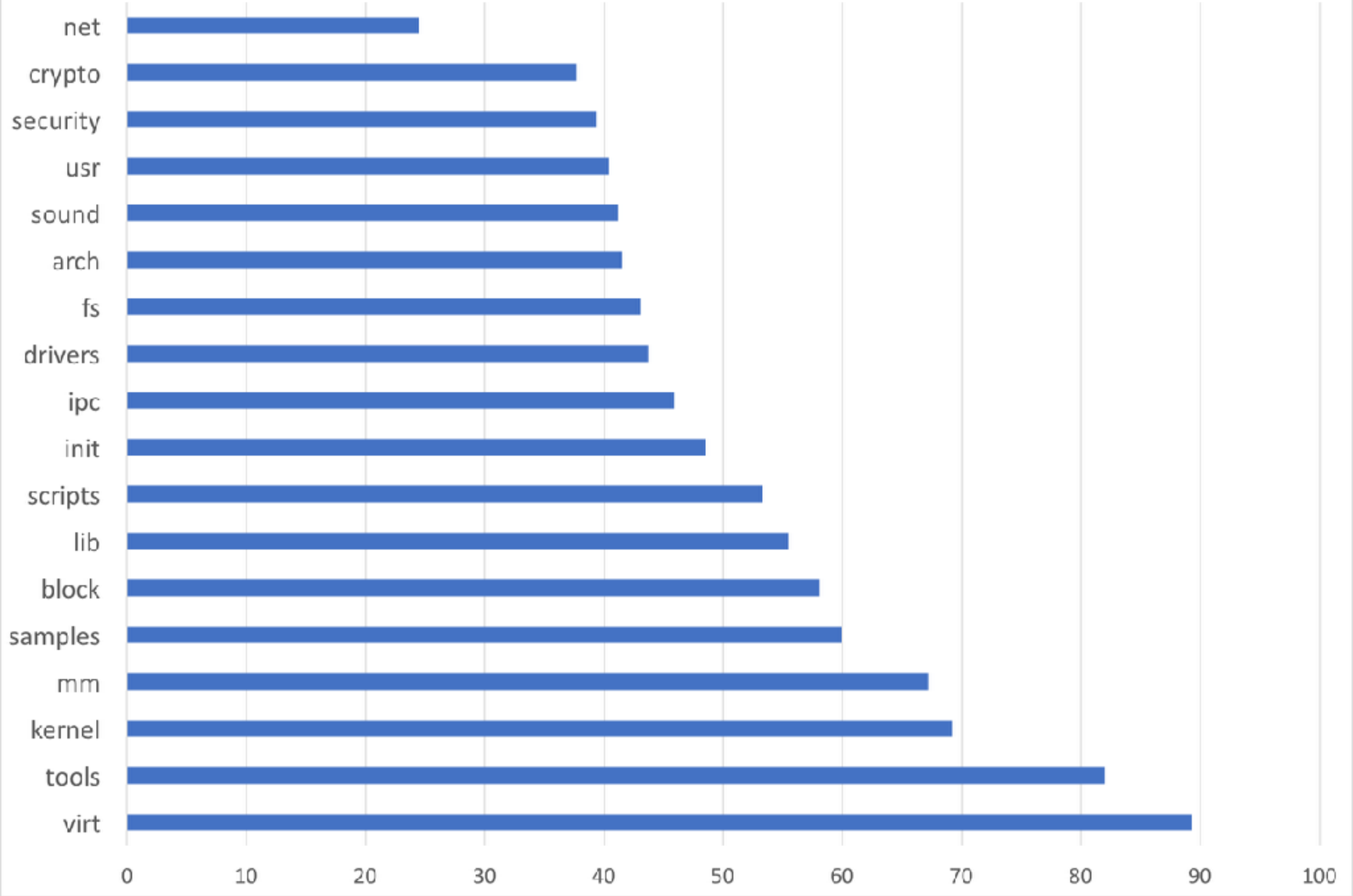
“fs/block_dev.c” → *[Patch 1, Patch 2, Patch 3, …]*



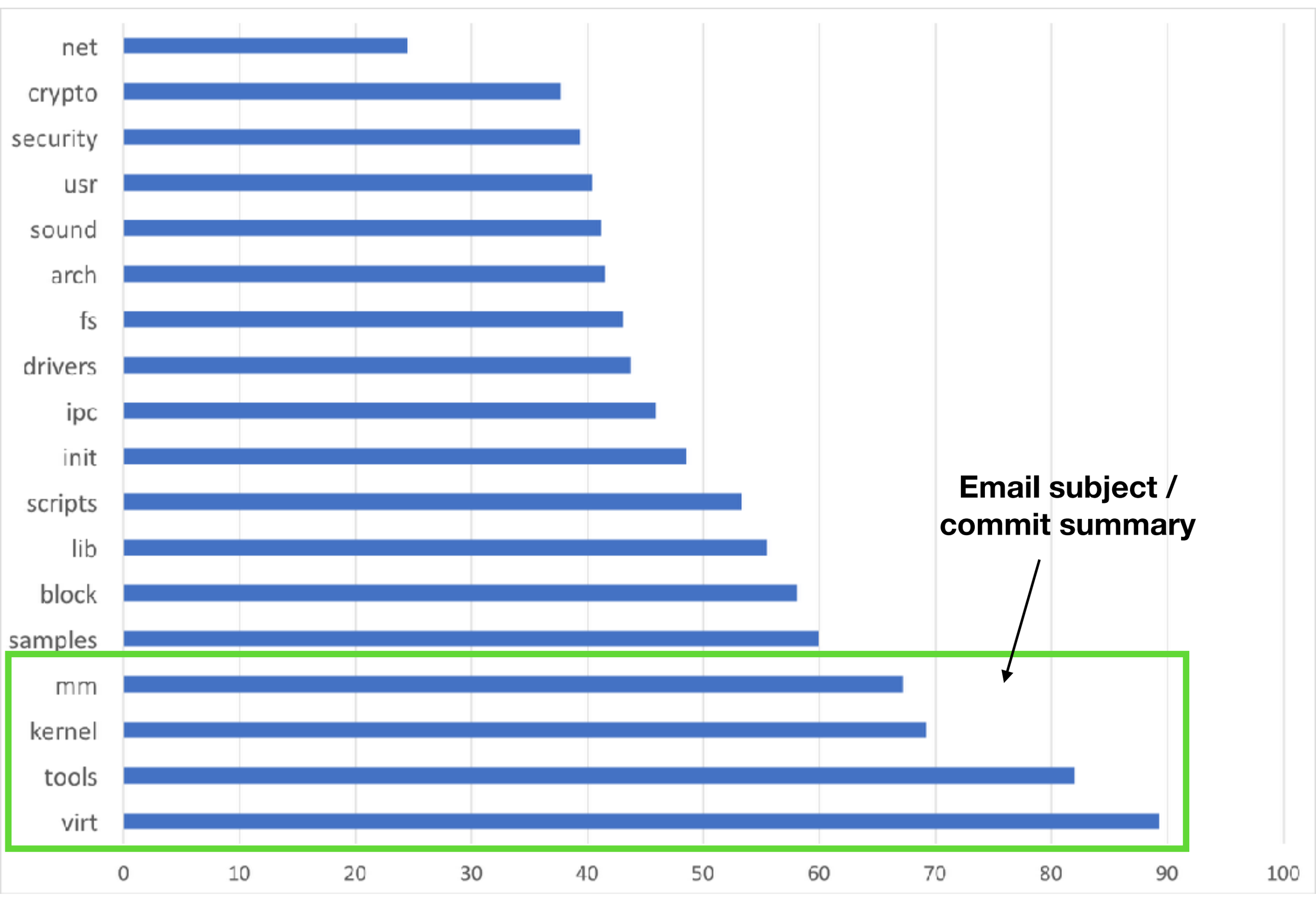
+/- line comparison

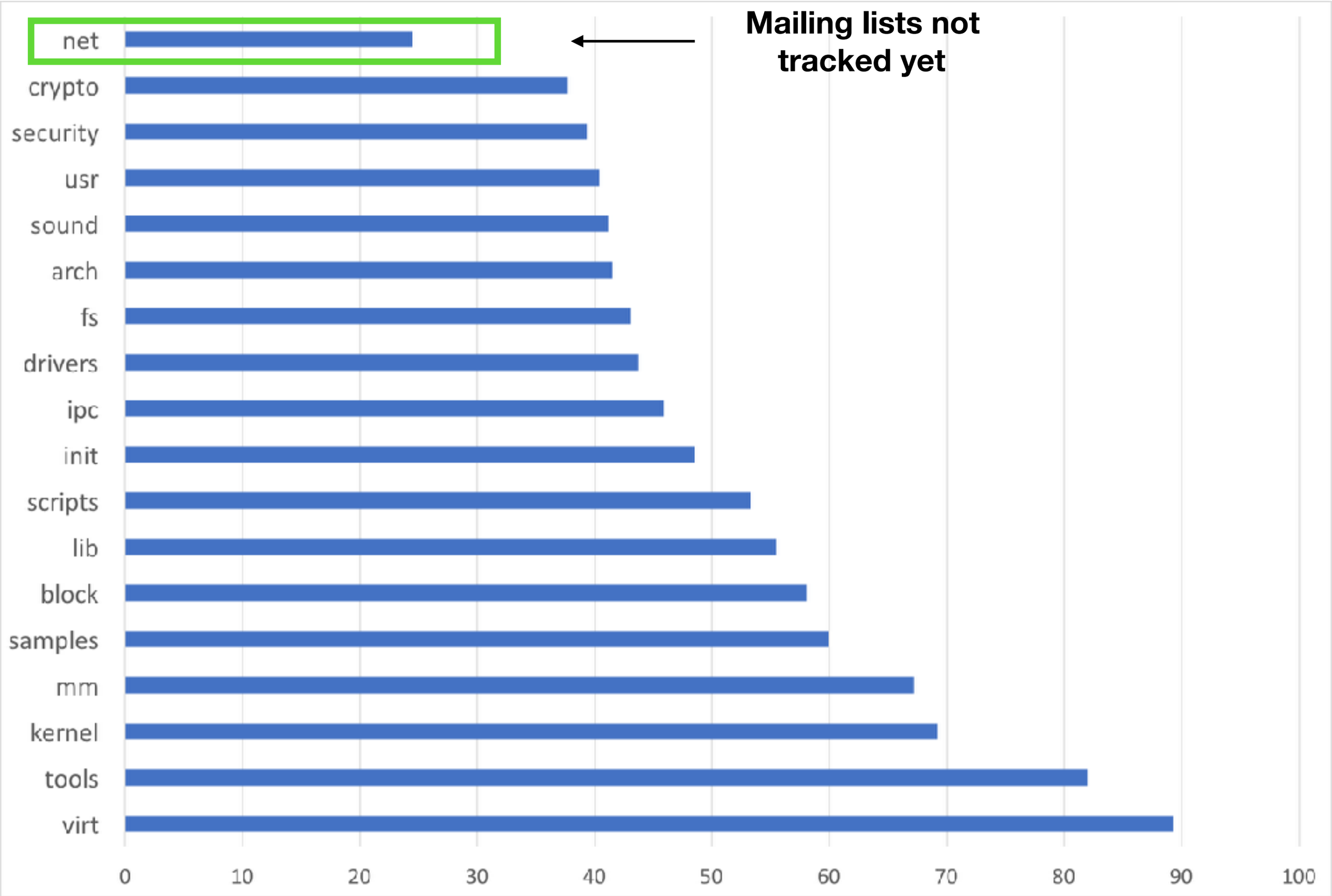
Commit - patch matches

How much have we matched?

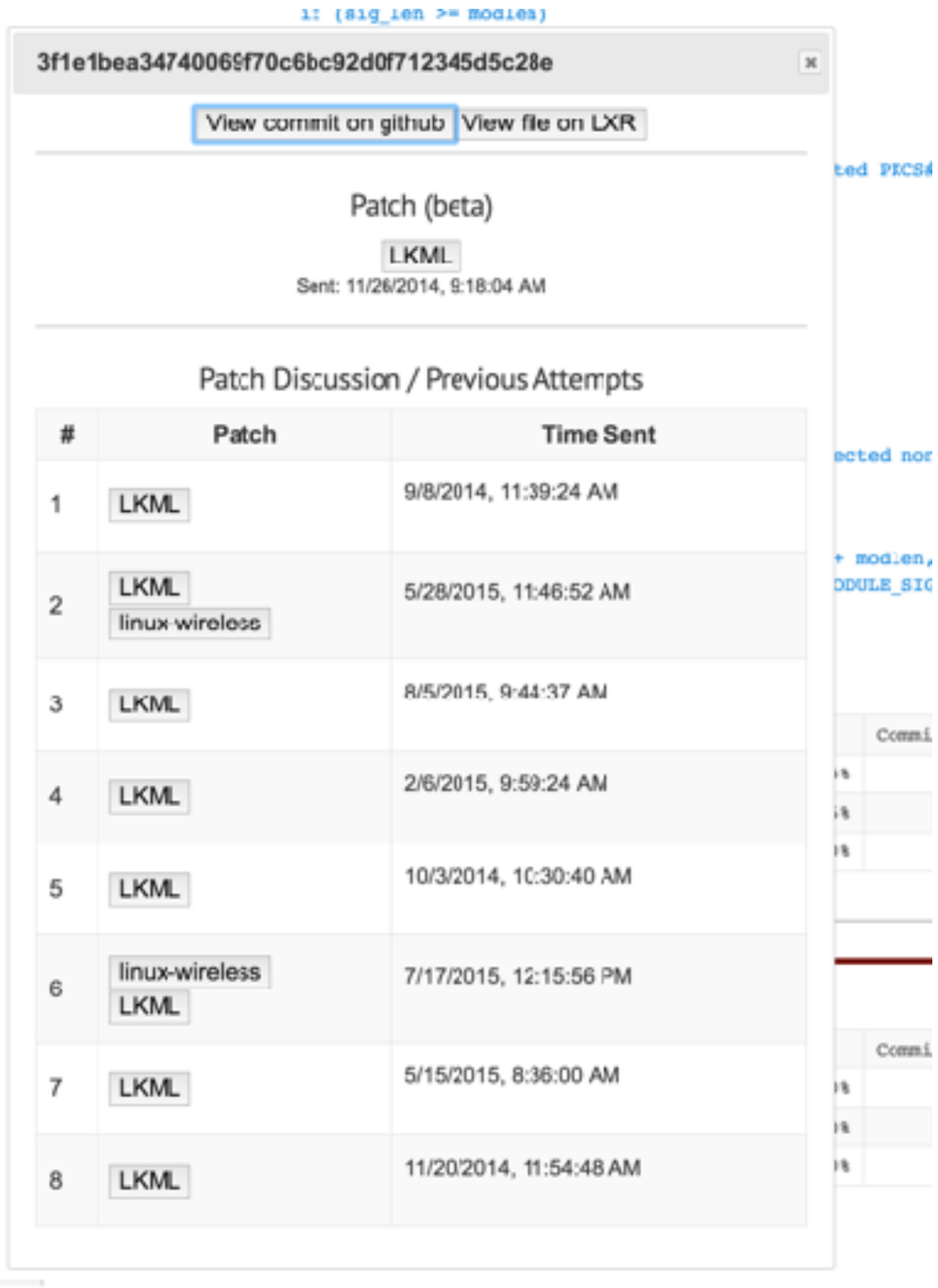


Percentage of **tokens matched** in 4.12


net
crypto
security
usr
sound
arch
fs
drivers
ipc
init
scripts
lib
block
samples
mm
kernel
tools
virt
0
10
20
30
40
50
60
70
80
90
100
Email subject / commit summary

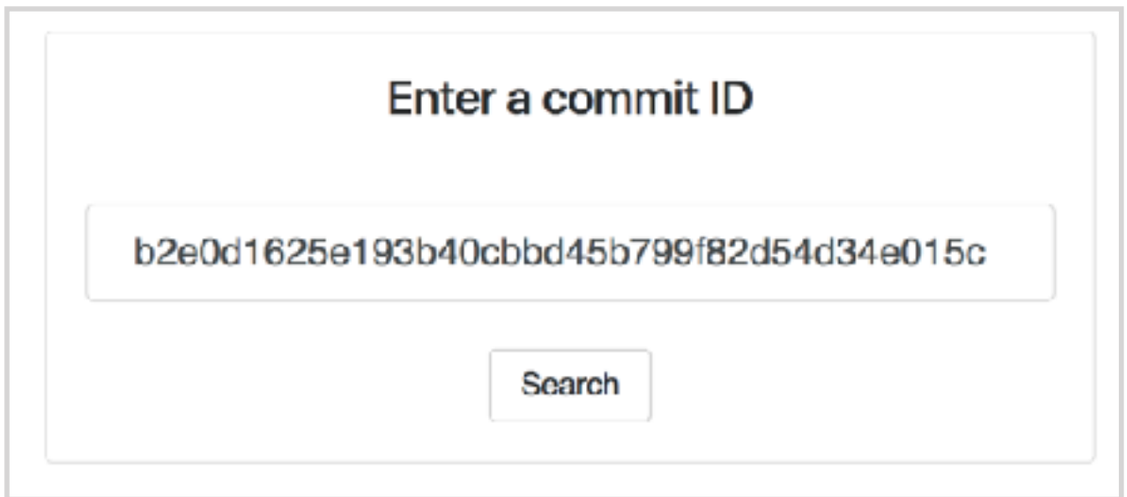

Mailing lists not tracked yet
net
crypto
security
usr
sound
arch
fs
drivers
ipc
init
scripts
lib
block
samples
mm
kernel
tools
virt
0
10
20
30
40
50
60
70
80
90
100

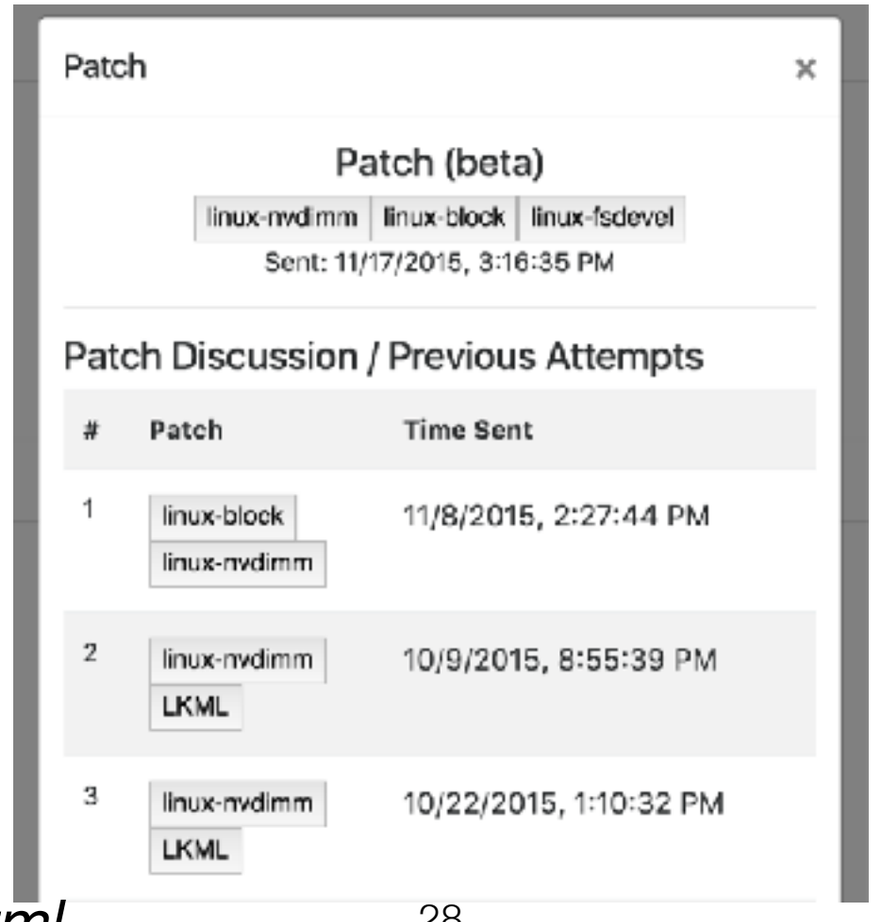
Accessing the Patches



Clicking on a token will fetch the patches for that token

Email2git interface





Limitations

Git

- Only tracking the main linux tree.
- Rebasing

Email

- Currently dependent on Patchwork (only patches sent from 2009)
- Mbox format inconsistent

Future directions

- Rest API to access matches
- Running our own instance of patchwork to parse mailing list archive “in-house” (extracting patch and comments)
- Track linux next
- Improve algorithm for incremental processing
- Patch series and multithreads
- Your feedback